



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه هشتم – حل مثال از تحلیل الگوریتم‌های بازگشتی، آرایه و مفاهیم آن

دکتر شهرام گلزاری

پاییز ۱۴۰۵

در پایان این جلسه انتظار داریم که:

- با **قضیه‌ی اصلی** زمان دو نسخه‌ی تابع `what` را تحلیل کنیم.
- رابطه‌ی بازگشتی **برج هانوی** را با **تکرار با جای‌گذاری** حل کنیم.
- نشانی عنصر $A[i]$ را در حافظه حساب کنیم و توضیح دهیم **چرا دسترسی به آن $O(1)$ است**.
- توضیح دهیم چرا وقتی عنصرها هم‌اندازه نیستند، آرایه **اشاره‌گر** نگه می‌دارد.
- مزیت‌های **آرایه‌ی مرتب** و هزینه‌ی درج در آن را بیان کنیم.

در جلسه ی قبل:

- رابطه ی بازگشتی مرتب سازی ادغامی را نوشتیم: $T(n) = 2T(n/2) + \Theta(n)$.
- با **درخت بازگشت** نشان دادیم $T(n) = \Theta(n \lg n)$.
- هفت روش حل رابطه ی بازگشتی را مرور کردیم.

در این جلسه دو روش از آن هفت روش را روی مثال به کار می بریم:

- قضیه ی اصلی **برای تابع what**،
- تکرار با جای گذاری **برای برج های هانوی**،
- و سپس سراغ نخستین داده ساختار درس، یعنی **آرایه**، می رویم.

مثال: تابع what

هر دو تابع چه چیزی را حساب می‌کنند؟ هر دو a^n را حساب می‌کنند.

```
WHAT2( $a, n$ )  
1 if  $n == 0$   
2   return 1  
3 if  $n == 1$   
4   return  $a$   
5  $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$   
6 if  $n$  is even  
7   return  $c \cdot c$   
8 else return  $a \cdot c \cdot c$ 
```

```
WHAT1( $a, n$ )  
1 if  $n == 0$   
2   return 1  
3 if  $n == 1$   
4   return  $a$   
5  $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$   
6  $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$   
7 if  $n$  is even  
8   return  $c \cdot d$   
9 else return  $a \cdot c \cdot d$ 
```

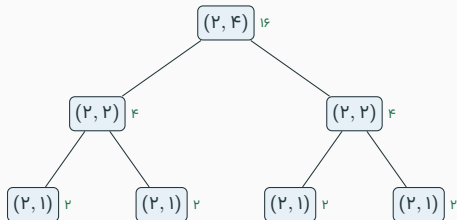
WHAT1 دو بار همان زیرمسئله را حل می‌کند ($c = d$)؛ WHAT2 یک بار حل می‌کند و جوابش را دو بار به کار می‌برد.

WHAT2



۳ فراخوانی

WHAT1



۷ فراخوانی

برای رابطه‌ی $T(n) = aT(n/b) + f(n)$ با ثابت‌های $a > 0$ و $b > 1$ ، تابع $f(n)$ را با $n^{\log_b a}$ مقایسه می‌کنیم:

حالت	شرط	جواب
۱	$f(n) = O(n^{\log_b a - \epsilon})$ برای یک $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
۲	$f(n) = \Theta(n^{\log_b a} \lg^k n)$ برای یک $k \geq 0$	$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
۳	$f(n) = \Omega(n^{\log_b a + \epsilon})$ برای یک $\epsilon > 0$ و $af(n/b) \leq cf(n)$ برای یک $c < 1$	$T(n) = \Theta(f(n))$

یادآوری: مرتب‌سازی ادغامی: $a = b = 2$ ، $n^{\log_2 2} = n$ و $f(n) = \Theta(n)$ حالت ۲ با $k = 0$ $\Theta(n \lg n)$.

بر اساس *CLRS, 4th ed.*، بخش ۴.۵، قضیه‌ی ۴.۱

WHAT2

```

WHAT2( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$ 
6  if  $n$  is even
7    return  $c \cdot c$ 
8  else return  $a \cdot c \cdot c$ 
    
```

- خط‌های ۱ تا ۴ و ۶ تا ۸: زمان ثابت، یعنی $\Theta(1)$.
- خط ۵: فقط یک فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $T(n/2)$.

$$T(n) = T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(\lg n)$$

WHAT1

```

WHAT1( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
6   $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
7  if  $n$  is even
8    return  $c \cdot d$ 
9  else return  $a \cdot c \cdot d$ 
    
```

- خط‌های ۱ تا ۴ و ۷ تا ۹: زمان ثابت، یعنی $\Theta(1)$.
- خط‌های ۵ و ۶: دو فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $2T(n/2)$.

$$T(n) = 2T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(n)$$

WHAT2

```

WHAT2( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT2}(a, \lfloor n/2 \rfloor)$ 
6  if  $n$  is even
7    return  $c \cdot c$ 
8  else return  $a \cdot c \cdot c$ 
    
```

- خط‌های ۱ تا ۴ و ۶ تا ۸: زمان ثابت، یعنی $\Theta(1)$.
- خط ۵: فقط یک فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $T(n/2)$.

$$T(n) = T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(\lg n)$$

WHAT1

```

WHAT1( $a, n$ )
1  if  $n == 0$ 
2    return 1
3  if  $n == 1$ 
4    return  $a$ 
5   $c = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
6   $d = \text{WHAT1}(a, \lfloor n/2 \rfloor)$ 
7  if  $n$  is even
8    return  $c \cdot d$ 
9  else return  $a \cdot c \cdot d$ 
    
```

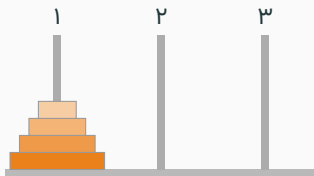
- خط‌های ۱ تا ۴ و ۷ تا ۹: زمان ثابت، یعنی $\Theta(1)$.
- خط‌های ۵ و ۶: دو فراخوانی روی $\lfloor n/2 \rfloor$ ، یعنی $2T(n/2)$.

$$T(n) = 2T(n/2) + \Theta(1)$$

کدام یک از حالت‌های قضیه اصلی است؟

$$T(n) = \Theta(n)$$

الگوریتم‌های بازگشتی: برج‌های هانوی



مثالی از مسئله‌ی برج‌های هانوی

«در این مسئله‌ی قدیمی و آشنا، n سکه‌ی سوراخ‌دار با قطرهای متفاوت به‌ترتیب قطرشان در میله‌ی شماره‌ی ۱ از ۳ میله‌ی موجود قرار دارند (سکه‌ی بزرگ‌تر در پایین سکه‌هاست). شکل ۳-۷ مثالی از حالت اولیه‌ی این مسئله را با ۴ سکه نشان می‌دهد. هدف، انتقال همه‌ی سکه‌ها به میله‌ی شماره‌ی ۳ به‌کمک میله‌ی شماره‌ی ۲ است به‌طوری‌که،

- هر بار فقط یک سکه جابه‌جا شود،
- در هر حرکت سکه‌ی کوچک‌تر بر روی سکه‌ی بزرگ‌تر قرار گیرد، و
- تعداد حرکت‌ها کمینه باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۸۴ و ۸۵ (بازترسیم شکل ۳-۷)

برای جابه‌جا کردن بزرگ‌ترین سکه، باید همه‌ی $n - 1$ سکه‌ی دیگر روی میله‌ی ۲ باشند. پس:



۱. $n - 1$ سکه‌ی بالایی را از میله‌ی ۱ به میله‌ی ۲ ببر .
۲. بزرگ‌ترین سکه را از میله‌ی ۱ به میله‌ی ۳ ببر .
۳. $n - 1$ سکه را از میله‌ی ۲ به میله‌ی ۳ ببر .

HA NOI(n, src, dst, tmp)

```
1 if  $n > 0$  then
2   |   HA NOI( $n - 1, src, tmp, dst$ )
3   |   move disk  $n$  from  $src$  to  $dst$ 
4   |   HA NOI( $n - 1, tmp, dst, src$ )
```

Erickson, Algorithms, صفحه ۲۶ (شکل ۱.۴)

اگر $T(n)$ تعداد حرکت‌ها باشد:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$

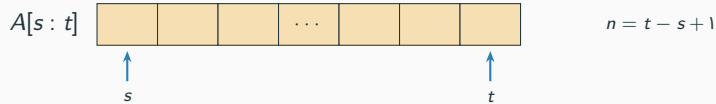
زیرمسئله‌ها $n - 1$ هستند، نه n/b ؛ پس قضیه‌ی اصلی این‌جا به کار نمی‌آید. از روش ۱، تکرار با جای‌گذاری، استفاده می‌کنیم.

$$\begin{aligned}
 T(n) &= 2T(n-1) + 1 \\
 &= 2[2T(n-2) + 1] + 1 = 2^2 T(n-2) + 2 + 1 \\
 &= 2^2 [2T(n-3) + 1] + 2 + 1 = 2^3 T(n-3) + 2^2 + 2 + 1 \\
 &\vdots \\
 &= 2^k T(n-k) + (2^{k-1} + \dots + 2 + 1) = 2^k T(n-k) + (2^k - 1)
 \end{aligned}$$

به حالت پایه می‌رسیم وقتی $n-k=1$ ، یعنی $k=n-1$:

$$T(n) = 2^{n-1} \cdot 1 + 2^{n-1} - 1 = 2^n - 1 = \Theta(2^n)$$

آرایه



برای شناختن یک آرایه این‌ها را باید بدانیم:

- **کران‌های اندیس:** اندیس نخستین عنصر s و اندیس آخرین عنصر t .
- **نوع عنصر** (element type): همه‌ی عنصرها از **یک نوع** اند.
- **اندازه‌ی نوع عنصر** (size of element type): هر عنصر b بایت جا می‌گیرد.
- **خانه‌های اشغال‌شده** (occupied): همه‌ی خانه‌ها **پشت سر هم** در حافظه‌اند.
- **نشانی شروع** a : نشانی نخستین بایت $A[s]$ در حافظه.

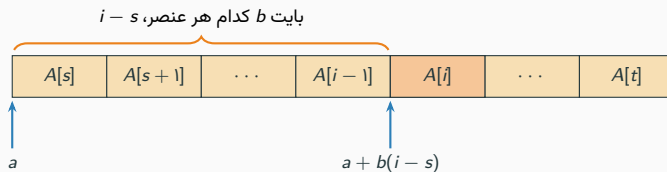
«فرض می‌کنیم که، مانند بیشتر زبان‌های برنامه‌نویسی، یک آرایه به صورت دنباله‌ای پیوسته از بایت‌ها در حافظه ذخیره می‌شود. اگر نخستین عنصر آرایه اندیس s داشته باشد (برای مثال، در آرایه‌ای با اندیس‌گذاری از 1 ، $s = 1$)، آرایه از نشانی حافظه‌ی a آغاز شود، و هر عنصر آرایه b بایت اشغال کند، آنگاه عنصر i ام بایت‌های $a + b(i - s)$ تا $a + b(i - s + 1) - 1$ را اشغال می‌کند.»

CLRS, 4th ed.، صفحه ۲۵۲ (ترجمه ماشینی)

«چون بیشتر آرایه‌های این کتاب از ۱ و چندتایی از ۰ اندیس‌گذاری شده‌اند، می‌توانیم این فرمول‌ها را کمی ساده کنیم. وقتی $s = 1$ ، عنصر i ام بایت‌های $a + b(i - 1)$ تا $a + bi - 1$ را اشغال می‌کند، و وقتی $s = 0$ ، عنصر i ام بایت‌های $a + bi$ تا $a + b(i + 1) - 1$ را اشغال می‌کند.»

CLRS, 4th ed.، صفحه ۲۵۲ (ترجمه ماشینی)

نخستین بایت $A[i]$ در کجای حافظه است؟ پیش از $A[i]$ دقیقاً $i - s$ عنصر قرار دارد و هر کدام b بایت است.



$$\text{addr}(A[i]) = a + b(i - s)$$

مثال: آرایه‌ی $A[۱۰ : ۳]$ از نوع `int`

فرض کنید هر `int` چهار بایت باشد ($b = ۴$)، $s = ۳$ ، و آرایه از نشانی $a = ۱۰۰۰$ شروع شود.

۱۰۰۰	۱۰۰۴	۱۰۰۸	۱۰۱۲	۱۰۱۶	۱۰۲۰	۱۰۲۴	۱۰۲۸
$A[۳]$	$A[۴]$	$A[۵]$	$A[۶]$	$A[۷]$	$A[۸]$	$A[۹]$	$A[۱۰]$

• تعداد عنصرها: $n = ۱۰ - ۳ + ۱ = ۸$ (روی هم $۸ \times ۴ = ۳۲$ بایت)

• نشانی $A[۷]$: $۱۰۰۰ + ۴(۷ - ۳) = ۱۰۱۶$ (بایت‌های ۱۰۱۶ تا ۱۰۱۹)

• نشانی $A[۱۰]$: $۱۰۰۰ + ۴(۱۰ - ۳) = ۱۰۲۸$

برای پیدا کردن $A[۷]$ لازم نشد از $A[۳]$ تا $A[۶]$ را ببینیم، **مستقیم** حساب کردیم.

چرا دسترسی به $A[i]$ از $O(1)$ است؟

- $A[1]$ در آرایه‌ی 10^6 عنصری: ۴ گام
- $A[10^6]$ در آرایه‌ی 10^6 عنصری: باز هم ۴ گام

دسترسی به $Arrayname[i]$ از $O(1)$ است.

برای خواندن $A[i]$:

گام	کار
۱	یک تفریق: $i - s$
۲	یک ضرب: $b(i - s)$
۳	یک جمع: $a + \dots$
۴	یک بار خواندن از حافظه در آن نشانی

تعداد گام‌ها همیشه ۴ است، نه به i بستگی دارد و نه به n .

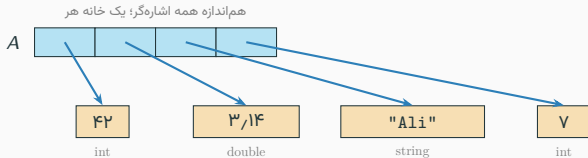
$O(1)$ بودن دسترسی روی سه پایه ایستاده است:

۱. خانه‌ها پشت سر هم اند $\Leftarrow$ نشانی را می‌شود با یک فرمول حساب کرد.
۲. عنصرها هم‌اندازه‌اند $\Leftarrow b$ ثابت است و فرمول ساده می‌ماند.
۳. حافظه دسترسی تصادفی (random access) دارد $\Leftarrow$ رفتن به هر نشانی زمان ثابت دارد.

اگر عناصر هم اندازه نباشند؟

«بیشتر زبان‌های برنامه‌نویسی لازم می‌دانند که همه‌ی عناصر یک آرایه‌ی مشخص هم‌اندازه باشند. اگر عناصر یک آرایه بتوانند تعداد بایت‌های متفاوتی اشغال کنند، فرمول‌های بالا به کار نمی‌آیند، زیرا اندازه‌ی عنصر b ثابت نیست. در چنین مواردی، عناصر آرایه معمولاً اشیائی با اندازه‌های متفاوت‌اند و آنچه در واقع در هر عنصر آرایه قرار می‌گیرد **یک اشاره‌گر به آن شیء** است. تعداد بایت‌هایی که یک اشاره‌گر اشغال می‌کند معمولاً یکسان است، فارغ از این‌که اشاره‌گر به چه چیزی اشاره کند؛ بنابراین برای دسترسی به یک شیء در آرایه، فرمول‌های بالا نشانی اشاره‌گر آن شیء را می‌دهند و سپس باید اشاره‌گر را دنبال کرد تا به خود شیء رسید.»

(ترجمه ماشینی) *CLRS, 4th ed.*، صفحه ۲۵۲



آرایه‌ی مرتب

آرایه‌ی $A[1 : n]$ مرتب است اگر

$$A[1] \leq A[2] \leq \dots \leq A[n]$$

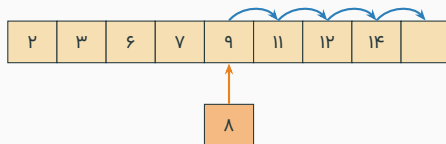
مثلاً خروجی MERGE-SORT در جلسه‌ی قبل:

۱	۲	۳	۴	۵	۶	۷	۸
۲	۳	۶	۷	۹	۱۱	۱۲	۱۴

مرتب بودن چه چیزی به ما می‌دهد؟

- کمینه $A[1]$ و بیشینه $A[n]$ است $\Leftarrow \Theta(1)$ به جای $\Theta(n)$.
- مقدارهای تکراری کنار هم‌اند $\Leftarrow$ پیدا کردنشان با یک پیمایش.
- جست‌وجو با جست‌وجوی دودویی در $\Theta(\lg n)$ به جای $\Theta(n)$.

می‌خواهیم ۸ را در آرایه‌ی مرتب درج کنیم و آرایه مرتب بماند:



- جای ۸ بین ۷ و ۹ است، پس ۹، ۱۱، ۱۲ و ۱۴ باید یک خانه به راست بروند.
 - چرا؟ چون خانه‌ها پشت سر هم اند، نمی‌توان بین دو خانه جا باز کرد.
 - بدترین حالت (درج کوچک‌ترین مقدار): هر n عنصر جابه‌جا می‌شوند $\Leftarrow \Theta(n)$.
- ویژگی پیوستگی که دسترسی به هر عنصر را $O(1)$ می‌کند، درج و حذف را پرهزینه می‌کند.

منابع

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022
- محمد قدسی — داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷
- Jeff Erickson — *Algorithms*, 2019